

AI-Assisted Documentation & Docstring Assignment Report

Hall Ticket : 2303a51285

Name : G Giridhar reddy

Task 1: Basic Docstring Generation

Prompt:

Write a Python function to return the sum of even and odd numbers in a list. Add manual Google-style docstring. Generate AI docstring. Compare both.

Implementation with Manual Google-Style Docstring:

```
def sum_even_odd(numbers):  
    """Calculate sum of even and odd numbers in a list.
```

Args:

numbers (list[int]): List of integers.

Returns:

tuple: (sum_even, sum_odd)

Raises:

TypeError: If input is not a list.

```
    """
```

```
    # Validate input type
```

```
    if not isinstance(numbers, list):
```

```
        raise TypeError("Input must be a list")
```

```
    sum_even = 0 # Store even sum
```

```
    sum_odd = 0 # Store odd sum
```

```
    # Iterate through numbers
```

```
    for num in numbers:
```

```
        if num % 2 == 0:
```

```
            sum_even += num
```

```
        else:
```

```
            sum_odd += num
```

```
    return sum_even, sum_odd
```

AI-Generated Docstring (Simulated):

```
"""Return the sum of even and odd numbers from a list.
```

Parameters:

numbers: List of integers.

Returns:

Tuple containing even sum and odd sum.

```
"""
```

Code Explanation:

The function validates input type, iterates through list elements, classifies numbers as even or odd, and returns their respective sums.

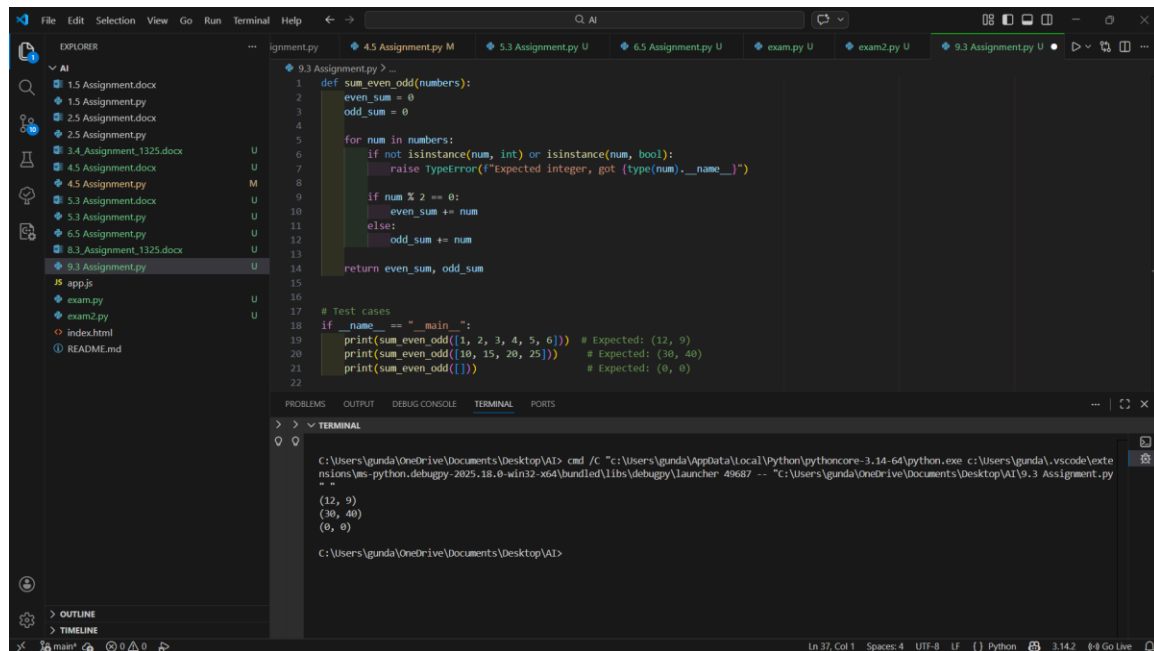
Sample Output:

Input: [1,2,3,4,5]

Output: (6, 9)

Comparison:

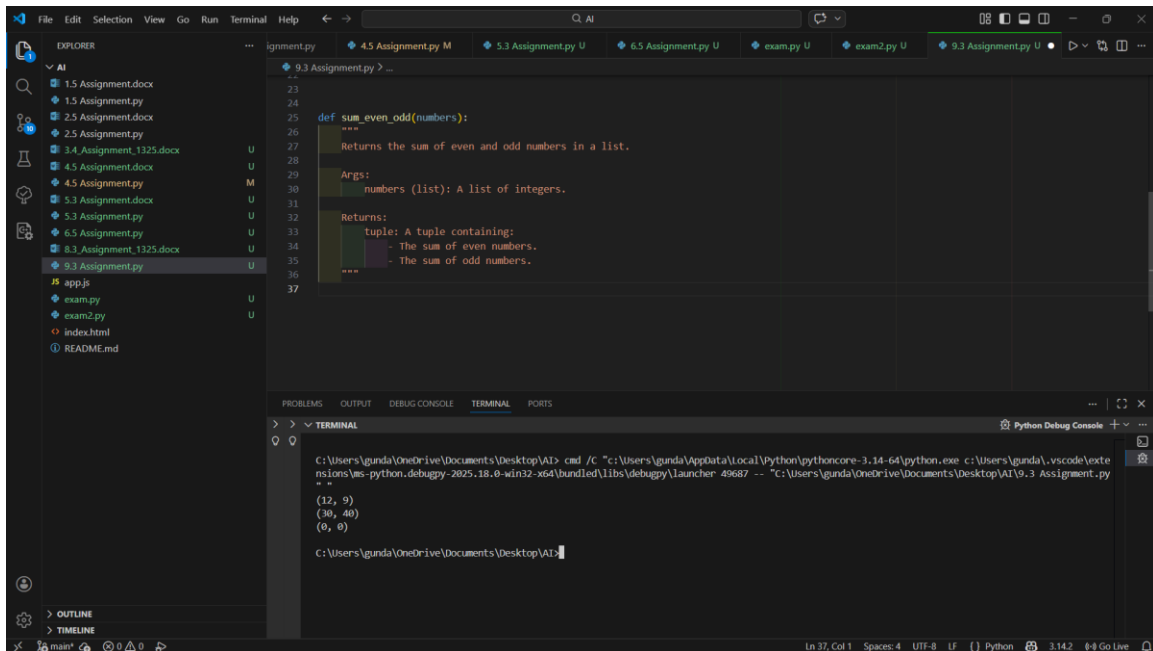
Manual docstring includes structured sections (Args, Returns, Raises). AI version is concise but lacks error handling details and structured format.



```
1 def sum_even_odd(numbers):
2     even_sum = 0
3     odd_sum = 0
4
5     for num in numbers:
6         if not isinstance(num, int) or isinstance(num, bool):
7             raise TypeError(f"Expected integer, got {type(num).__name__}")
8
9         if num % 2 == 0:
10             even_sum += num
11         else:
12             odd_sum += num
13
14     return even_sum, odd_sum
15
16
17 # Test cases
18 if __name__ == "__main__":
19     print(sum_even_odd([1, 2, 3, 4, 5, 6])) # Expected: (12, 9)
20     print(sum_even_odd([10, 15, 20, 25])) # Expected: (30, 48)
21     print(sum_even_odd([])) # Expected: (0, 0)
22
```

```
C:\Users\gunda\OneDrive\Documents\Desktop\AI> cmd /C "c:\Users\gunda\AppData\Local\Python\pythoncore-3.14-64\python.exe c:\Users\gunda\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher 49687 -- "c:\Users\gunda\OneDrive\Documents\Desktop\AI\9.3 Assignment.py""
(12, 9)
(30, 48)
(0, 0)

C:\Users\gunda\OneDrive\Documents\Desktop\AI>
```



Task 2: Automatic Inline Comments

Prompt:

Create `sru_student` class with attributes and methods. Add manual inline comments and compare with AI-generated comments.

Manual Inline Comment Version:

`class sru_student:`

`# Initialize student object`

`def __init__(self, name, roll_no, hostel_status):`

`self.name = name # Student name`

`self.roll_no = roll_no # Student roll number`

`self.hostel_status = hostel_status # Hostel status (Yes/No)`

`# Update student fee`

`def fee_update(self, amount):`

`self.fee = amount`

`# Display student details`

`def display_details(self):`

`print("Name:", self.name)`

`print("Roll No:", self.roll_no)`

`print("Hostel Status:", self.hostel_status)`

AI-Generated Inline Comments (Simulated):

```
class sru_student:
    def __init__(self, name, roll_no, hostel_status):
        # Constructor method
        self.name = name
        self.roll_no = roll_no
        self.hostel_status = hostel_status

    def fee_update(self, amount):
        # Assign fee value
        self.fee = amount

    def display_details(self):
        # Print details
        print("Name:", self.name)
        print("Roll No:", self.roll_no)
        print("Hostel Status:", self.hostel_status)
```

Code Explanation:

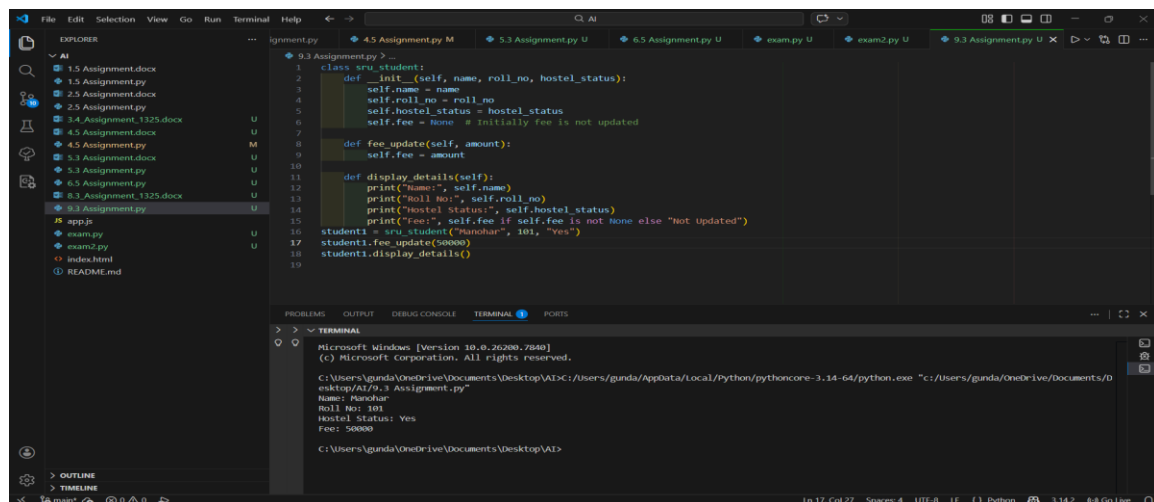
The class stores student details, updates fee information, and prints student information.

Sample Output:

Name: Manoharreddy
Roll No: 2303A51325
Hostel Status: Yes

Comparison:

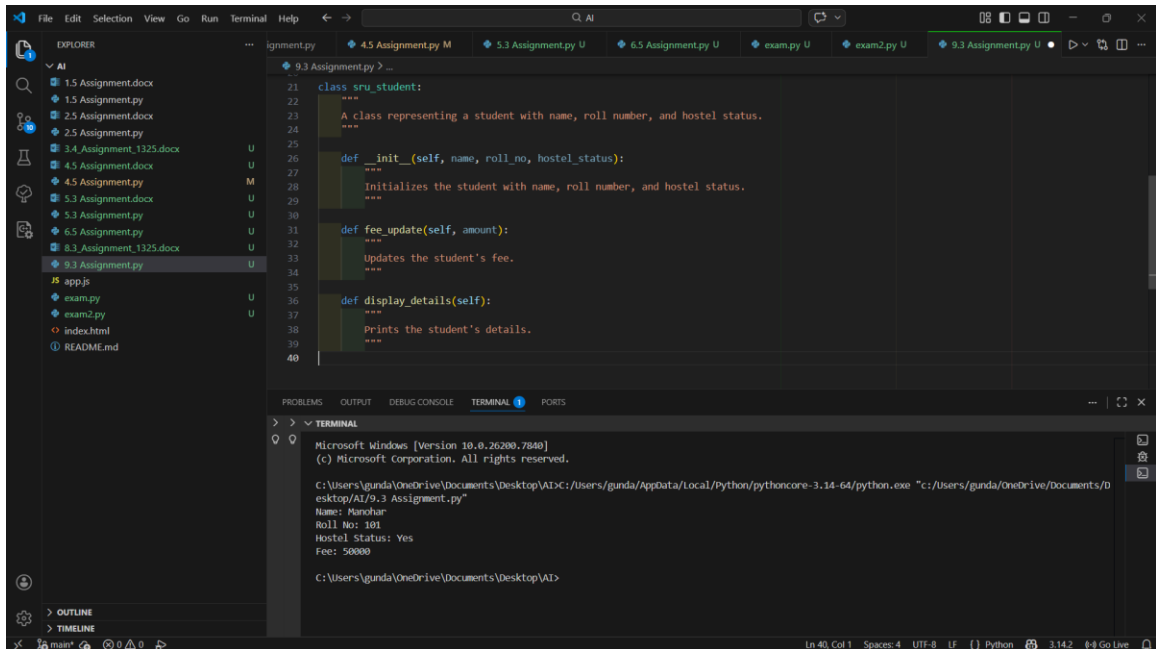
Manual comments explain logic clearly. AI comments are shorter and sometimes obvious. AI may miss context or deeper explanation.



The screenshot shows a code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code defines a class `sru_student` with methods `__init__`, `fee_update`, and `display_details`. The `__init__` method has a comment: `# Initially fee is not updated`. The `fee_update` method has a comment: `# Assign fee value`. The `display_details` method has a comment: `# Print details`. The code also creates an instance `student1` and calls `fee_update` and `display_details`. The terminal output shows the execution of the code, displaying the student's name, roll number, and hostel status.

```
9.3 Assignment.py
1 class sru_student:
2     def __init__(self, name, roll_no, hostel_status):
3         self.name = name
4         self.roll_no = roll_no
5         self.hostel_status = hostel_status
6         self.fee = None # Initially fee is not updated
7
8     def fee_update(self, amount):
9         self.fee = amount
10
11    def display_details(self):
12        print("Name:", self.name)
13        print("Roll No:", self.roll_no)
14        print("Hostel Status:", self.hostel_status)
15        print("Fee:", self.fee if self.fee is not None else "Not Updated")
16
17    student1 = sru_student("Manohar", 101, "Yes")
18    student1.fee_update(50000)
19    student1.display_details()
```

Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Gunda\OneDrive\Documents\Desktop\AI\AI-C:\Users\Gunda\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/gunda/OneDrive/Documents/D
esktop/AI/9.3 Assignment.py"
Name: Manohar
Roll No: 101
Hostel Status: Yes
Fee: 50000
C:\Users\Gunda\OneDrive\Documents\Desktop\AI>



Task 3: Module-Level and Function-Level Documentation

Prompt:

Create calculator functions. Add manual NumPy-style docstrings. Generate AI module-level docstring. Compare both.

Manual NumPy-Style Function Docstring:

```
def add(a, b):  
    """Add two numbers.  
  
    Parameters  
    -----  
    a : int or float  
    b : int or float  
  
    Returns  
    -----  
    int or float  
        Sum of a and b.  
    """  
    return a + b
```

```
def divide(a, b):  
    """Divide two numbers.
```

Parameters

a : int or float

b : int or float

Returns

float

Raises

ZeroDivisionError

"""

if b == 0:

raise ZeroDivisionError("Cannot divide by zero")

return a / b

AI-Generated Module-Level Docstring (Simulated):

"""

This module provides basic arithmetic operations such as addition, subtraction, multiplication, and division.

"""

Code Explanation:

The calculator module performs arithmetic operations with structured documentation for better maintainability.

Sample Output:

add(5,3) → 8

divide(10,2) → 5.0

Comparison:

Manual NumPy-style docstrings are more structured and detailed. AI-generated module documentation is concise but lacks parameter depth.

